

SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – CONSTRAINT-BASED PROBLEM SOLVING

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO5 – Precision (BL4)	Assessment:	Constraint Based Problem Solving
Weightage:	40%	Total Marks:	100
CO5 Statement:	Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.		
Student Name:	SIVARANGJENII B	Reg. No.:	192521375
Section / Batch:	B.TECH IT	Date:	25.8.26

Instructions to Students

- Identify the relevant concepts of I/O interfaces, interrupts, DMA, bus arbitration, and high-speed communication protocols.
- Analyze the I/O requirements considering CPU utilization, latency, throughput, bandwidth, and device priorities.
- Apply suitable I/O techniques such as DMA, vectored interrupts, memory-mapped I/O, nested interrupts, and bus arbitration.
- Compute performance measures such as data transfer rate, interrupt latency, CPU utilization, throughput, and transfer time.
- Interpret the results by evaluating CPU efficiency, communication performance, interrupt response, and suitability of the proposed I/O architecture.

QUESTIONS

1. Construct an I/O communication mechanism for a real-time embedded system with the following constraints:

- CPU utilization must be below 15%
- Multiple sensors generate interrupts every 1 ms
- Use vectored interrupt handling
 - Select between memory-mapped and I/O-mapped I/O for optimal latency

2. Design an I/O subsystem under these constraints:

- High-speed storage requires throughput > 3 GB/s
- Must use NVMe over PCIe
- CPU should not handle bulk data transfer
- Integrate DMA controller with interrupt notification

3. Construct a multi-device communication architecture with constraints:

- 6 peripherals using USB, 2 high-speed devices using Thunderbolt
- Avoid interrupt starvation
- Use hardware and software interrupts
- Ensure fair bus arbitration

4. Design a data acquisition system with constraints:

Marks Evaluation:

Q. No	Problem Understanding (25)	Constraint Analysis (25)	Solution Development (25)	Application of Concepts (15)	Justification (10)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/25	/ 25	/ 15	/ 10	/ 100

Course Coordinator Faculty Incharge

1. I/O Communication Mechanism for a Real-Time Embedded System

Problem Understanding — 25%

The objective is to construct an I/O communication mechanism for a real-time embedded system where multiple sensors generate interrupts every 1 ms. Since the system is real-time, the CPU must respond to sensor events quickly while keeping CPU utilization below 15%. The system must also use vectored interrupt handling so that each interrupt can directly access its corresponding service routine.

The main design decision is to select between memory-mapped I/O (MMIO) and I/O-mapped I/O. The selected method should provide low-latency communication between the CPU and sensor devices while minimizing CPU overhead.

Given constraints

- CPU utilization $< 15\%$
- Multiple sensors generate interrupts every 1 ms
- Vectored interrupt handling is mandatory
- I/O mechanism must provide low latency
- Need to select between memory-mapped I/O and I/O-mapped I/O

Constraint Analysis — 25%

A. CPU utilization below 15%

If every sensor interrupt requires significant CPU processing, the CPU may spend too much time servicing interrupts.

Therefore, the interrupt service routine (ISR) should be:

- Very short
- Fast
- Limited to reading sensor status/data
- Responsible for setting a flag or placing data into a buffer

- Followed by normal/background processing outside the ISR

This reduces the amount of CPU time spent inside interrupt handlers.

B. Sensors generate interrupts every 1 ms

A 1 ms interval means:

So each sensor can generate approximately 1000 interrupts per second.

If there are sensors:

For example, with 5 sensors:

Therefore, the interrupt mechanism must be efficient enough to handle a high interrupt rate.

C. Vectored interrupt handling

Each sensor should have a separate interrupt vector.

Instead of checking every sensor to determine which one generated the interrupt, the CPU directly jumps to the appropriate ISR using the interrupt vector.

This reduces interrupt-identification overhead and improves response time.

D. MMIO vs I/O-mapped I/O

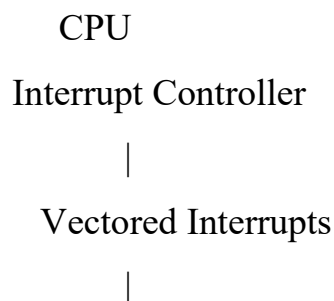
Memory-mapped I/O maps sensor registers into the CPU's memory address space. The CPU can access the registers using normal load/store instructions.

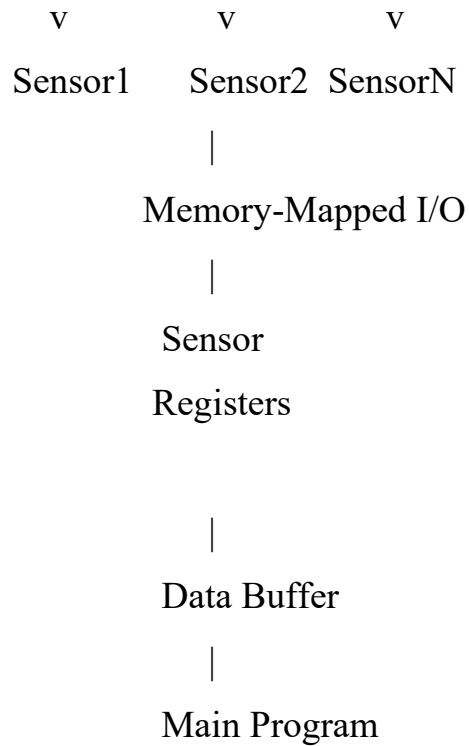
I/O-mapped I/O uses a separate I/O address space and dedicated I/O instructions.

For this real-time system, memory-mapped I/O is selected because it provides a simple and efficient register-access mechanism and integrates well with modern embedded processors.

Solution Development — 25%

Proposed architecture





Working mechanism

Step 1: Each sensor monitors its physical parameter continuously.

Step 2: When a sensor requires CPU attention, it generates an interrupt.

Step 3: The interrupt controller identifies the sensor and provides its interrupt vector.

Step 4: The CPU immediately jumps to the corresponding ISR.

Step 5: The ISR reads the sensor status/data through memory-mapped registers.

Step 6: The ISR stores the data in a buffer and sets a processing flag.

Step 7: The ISR exits quickly.

Step 8: The main application processes the buffered sensor data outside the ISR.

This prevents lengthy processing from blocking other real-time interrupts.

CPU utilization analysis

CPU utilization caused by interrupts can be approximately expressed as:

where:

- = number of sensors
- = interrupt frequency per sensor
- = CPU cycles/time required by ISR

- = total CPU cycles/time available

The design keeps the ISR very short so that:

The remaining CPU capacity is available for application processing and other system tasks.

Application of Concepts — 15%

The following embedded-system concepts are applied:

Vectored interrupts

Each sensor is assigned a unique interrupt vector.

Sensor 1 → Vector 1 → ISR 1

Sensor 2 → Vector 2 → ISR 2

Sensor 3 → Vector 3 → ISR 3

This avoids unnecessary polling and reduces interrupt response time.

Memory-mapped I/O

Sensor registers are assigned memory addresses.

Sensor 1 Status → 0x8000

Sensor 1 Data → 0x8004

Sensor 2 Status → 0x8010

Sensor 2 Data → 0x8014

The CPU can directly access these registers using normal memory operations.

Interrupt controller

An interrupt controller receives interrupts from all sensors and manages:

- Interrupt priority
- Vector generation
- Interrupt masking
- Interrupt delivery to the CPU

Short ISR

The ISR performs only essential operations:

Receive interrupt



Identify vector



Read sensor register



Store data in buffer



Set processing flag



Return from interrupt

Long calculations are avoided inside the ISR.

Justification — 10%

Memory-mapped I/O with vectored interrupts is selected as the optimal solution.

Vectored interrupt handling provides fast identification of the interrupt source because the CPU directly jumps to the corresponding ISR. This is important because sensors can generate interrupts every 1 ms.

Memory-mapped I/O provides efficient access to sensor registers using normal CPU instructions. Keeping the ISR short minimizes CPU overhead and helps maintain CPU utilization below the required 15%.

The architecture also uses buffering and deferred processing so that the CPU does not perform lengthy operations during interrupt servicing.

Therefore, the proposed mechanism satisfies all the given constraints:

Constraint	Proposed solution
CPU utilization < 15%	Short ISR + deferred processing
Sensors interrupt every 1 ms	Fast interrupt controller + vectored interrupts
Vectored interrupt handling	Separate vector for each sensor
Low latency	Direct vector-based ISR dispatch
I/O selection	Memory-mapped I/O
Multiple sensors	Central interrupt controller + buffering

Final Design

Sensors → Interrupt Controller → Vectored ISR → Memory-Mapped Sensor Registers
→ Buffer → Main Application

This design provides low-latency, efficient and scalable I/O communication for the real-time embedded system while satisfying the required CPU utilization constraint.

Q2. I/O Subsystem for High-Speed Storage

Problem Understanding

The objective is to design an I/O subsystem for a high-speed storage device that provides a throughput of more than 3 GB/s. The system must use NVMe over PCIe, and the CPU should not be responsible for bulk data transfer. Therefore, DMA (Direct Memory Access) must be used to transfer data directly between the storage device and main memory.

The DMA controller should also generate an interrupt notification after completing a transfer so that the CPU can perform necessary control operations without continuously monitoring the data transfer.

Constraint Analysis

The storage subsystem must achieve a throughput greater than:

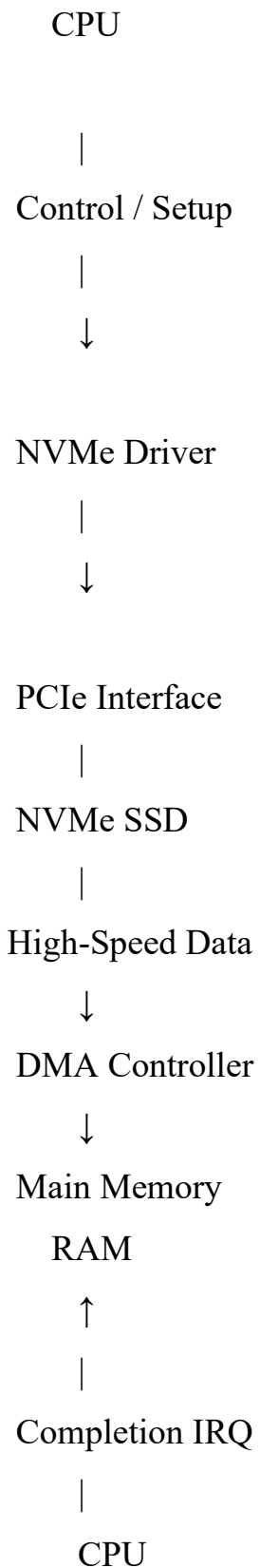
Therefore, a high-bandwidth interface such as PCIe is required.

NVMe is specifically designed for high-speed solid-state storage and uses PCIe to communicate with the system.

The CPU should not handle the actual movement of large blocks of storage data because this would consume CPU resources. Instead, DMA should transfer data directly between the NVMe device and system memory.

The CPU should only configure the DMA operation and respond to completion notifications.

Proposed I/O Architecture



Working of the Proposed System

The CPU first initializes the NVMe driver and configures the DMA controller with the source address, destination address and amount of data to transfer.

The NVMe SSD communicates with the system through the PCIe interface.

When a large block of data must be transferred, the DMA controller performs the transfer directly between the NVMe device and main memory.

The CPU does not copy every byte or word of data. It only configures the transfer and performs control operations.

After the DMA transfer is completed, the DMA controller generates an interrupt. The CPU receives the interrupt and the NVMe driver checks the completion status.

CPU configures DMA



NVMe request issued



PCIe transfers data



DMA moves data to RAM



DMA transfer completed



Interrupt generated



CPU handles completion

Throughput Analysis

The required storage throughput is:

The PCIe-based NVMe architecture is selected because it provides significantly higher bandwidth than traditional storage interfaces.

The actual achievable throughput depends on the PCIe generation, number of lanes, NVMe SSD capability and system implementation.

The design therefore targets a PCIe/NVMe configuration capable of providing more than 3 GB/s effective storage throughput.

Application of DMA

DMA allows the storage device to transfer large amounts of data directly into RAM.

Without DMA:

NVMe → CPU → RAM

The CPU would participate heavily in data movement.

With DMA:

NVMe → DMA Controller → RAM

The CPU is removed from the bulk data path.

This significantly reduces CPU overhead and allows the CPU to execute other tasks while the transfer is in progress.

Interrupt Notification

After completing the DMA transfer, the DMA controller generates an interrupt.

The interrupt can indicate:

- Transfer completed
- Transfer error
- Invalid memory address
- Device error

The CPU then executes the appropriate interrupt service routine.

This is more efficient than continuously polling the DMA controller.

Justification

NVMe over PCIe is selected because the requirement is greater than 3 GB/s, which requires a high-speed storage interface.

DMA is used because the CPU should not handle bulk data transfer. The DMA controller directly transfers data between the NVMe device and system memory.

Interrupt notification allows the CPU to know when the transfer has completed without continuously checking the device.

Therefore, the proposed architecture provides high throughput, low CPU overhead and efficient storage communication.

Constraint Satisfaction

Constraint	Proposed Solution
Throughput > 3 GB/s	High-speed NVMe over PCIe
Must use NVMe over PCIe	NVMe SSD connected through PCIe
CPU should not handle bulk transfer	DMA performs data movement
DMA controller required	DMA directly transfers data to/from RAM
Interrupt notification	DMA generates completion interrupt
Efficient CPU utilization	CPU only handles setup and completion

Conclusion

The proposed NVMe-over-PCIe subsystem with DMA and interrupt notification satisfies the high-speed storage requirements. DMA removes bulk data movement from the CPU, while interrupt notification provides efficient completion handling. The architecture is therefore suitable for storage systems requiring throughput above 3 GB/s.

Q3. Multi-Device Communication Architecture

Problem Understanding

The objective is to construct a communication architecture supporting six USB

peripherals and two high-speed Thunderbolt devices. The system must prevent interrupt starvation, use both hardware and software interrupts, and provide fair bus arbitration.

Since multiple devices can request system resources at the same time, the architecture requires an interrupt controller and a bus arbitration mechanism that ensures high-priority devices receive timely service without completely blocking lower-priority devices.

Constraint Analysis

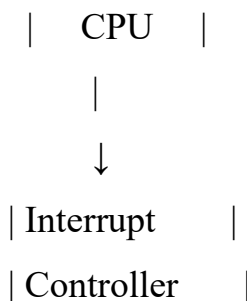
The system contains:

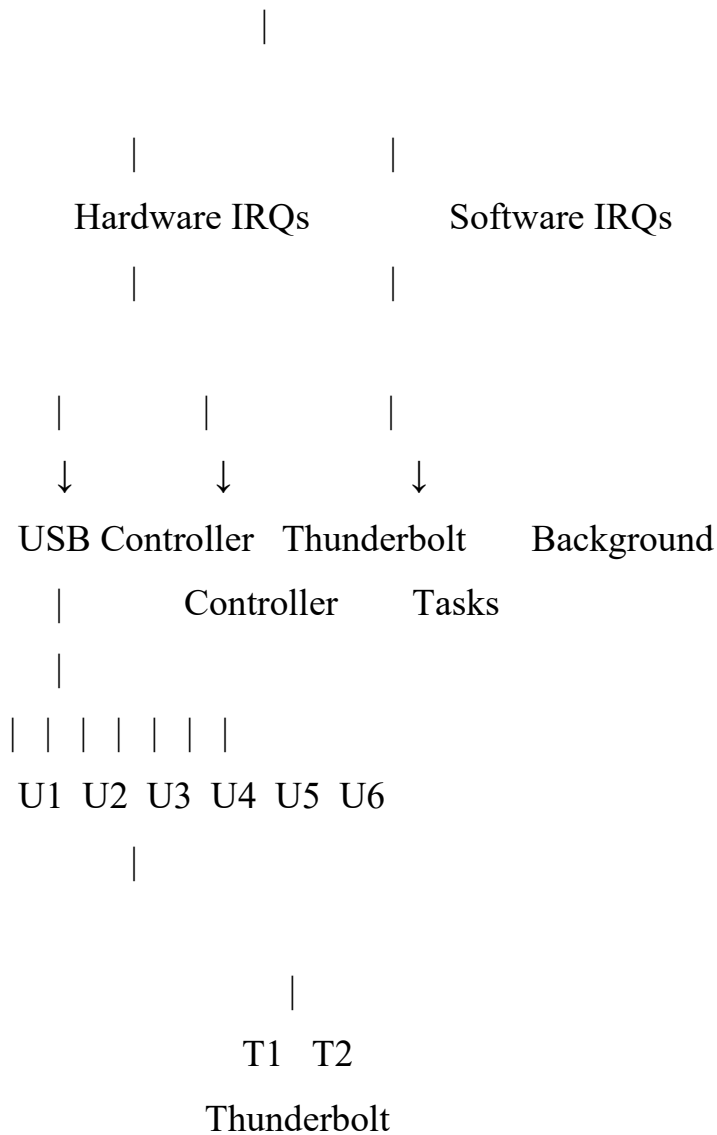
- 6 USB peripherals
- 2 Thunderbolt high-speed devices
- Hardware interrupts
- Software interrupts
- Multiple simultaneous bus requests
- Requirement to avoid interrupt starvation
- Requirement for fair bus arbitration

Thunderbolt devices require high-speed communication, while USB devices may generate interrupts for events such as data availability or transfer completion.

A fixed-priority system could cause low-priority USB devices to wait indefinitely if high-speed devices continuously generate requests. Therefore, priority with fairness mechanisms, such as round-robin arbitration or priority aging, should be used.

Proposed Communication Architecture





Working of the Proposed System

The six USB peripherals communicate through a USB host controller. The two Thunderbolt devices communicate through the Thunderbolt controller.

When a device requires CPU attention, it generates a hardware interrupt.

The interrupt controller receives the requests and determines which interrupt should be serviced.

Software interrupts are used for operating-system-level services and background processing.

Bus arbitration determines which device receives access to the shared communication resources.

Hardware Interrupts

Hardware interrupts are generated by physical devices.

Examples include:

USB device → Hardware IRQ

Thunderbolt device → Hardware IRQ

These interrupts are useful for notifying the CPU about events such as:

- Data arrival
- Transfer completion
- Device status changes
- Communication errors

Software Interrupts

Software interrupts are generated by software rather than directly by hardware.

They can be used for:

- Operating system services
- Deferred processing
- Background communication tasks
- Driver requests

This separates immediate hardware event handling from software-level processing.

Avoiding Interrupt Starvation

Interrupt starvation occurs when a low-priority device waits indefinitely because higher-priority devices continuously generate interrupts.

To prevent this, the system can use priority aging.

A waiting interrupt gradually receives higher priority as its waiting time increases.

For example:

New high-priority request



Served immediately



Low-priority request waits



Waiting time increases



Priority increases



Low-priority request gets service

This guarantees that lower-priority devices eventually receive CPU service.

Fair Bus Arbitration

A round-robin arbitration mechanism can be used to provide fair access to the shared bus.

USB1 → USB2 → USB3 → USB4 → USB5 → USB6



The Thunderbolt devices can be given sufficient service slots because of their high bandwidth requirements, while USB devices are still guaranteed access.

A weighted round-robin approach can also be used if Thunderbolt devices require more bandwidth.

Justification

The architecture separates USB and Thunderbolt communication using their respective controllers while using a centralized interrupt controller for CPU notification.

Hardware interrupts provide fast notification of physical device events. Software interrupts support operating-system and background operations.

Priority aging prevents interrupt starvation, while fair bus arbitration ensures that no device permanently monopolizes the communication channel.

Constraint Satisfaction

Constraint	Proposed Solution
6 USB peripherals	USB host controller
2 Thunderbolt devices	Thunderbolt controller
Avoid interrupt starvation	Priority aging

Constraint	Proposed Solution
Hardware interrupts	Device-generated IRQs
Software interrupts	OS/background service requests
Fair bus arbitration	Round-robin/weighted round-robin
High-speed communication	Dedicated Thunderbolt controller

Conclusion

The proposed architecture provides efficient communication for six USB peripherals and two Thunderbolt devices. Combining hardware and software interrupts with priority aging and fair bus arbitration prevents starvation and ensures that all devices receive appropriate system resources.

Q4. Data Acquisition System

Problem Understanding

The objective is to design a data acquisition system capable of continuously receiving data at 500 MB/s while requiring minimal CPU intervention.

The system must compare DMA transfer with interrupt-driven I/O and use memory-mapped I/O for accessing device registers.

Since the data stream is continuous and has a high rate of 500 MB/s, the system should avoid requiring the CPU to handle every data transfer event.

Constraint Analysis

The required data rate is:

This means that the system must continuously receive approximately 500 MB of data every second.

If the CPU were interrupted for every small data unit, the number of interrupts could become extremely high.

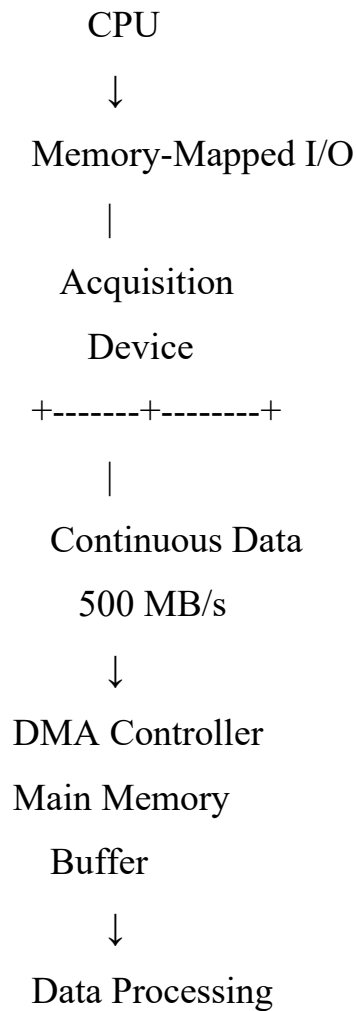
Therefore, a DMA-based architecture is more suitable because DMA can transfer blocks of data directly from the acquisition device to memory.

Memory-mapped I/O is used for controlling device registers such as:

- Control register
- Status register
- Data register

- DMA configuration registers

Proposed Data Acquisition Architecture



Working of the Proposed System

The CPU first configures the acquisition device using memory-mapped registers.

The acquisition device continuously produces data at 500 MB/s.

Instead of sending every data unit to the CPU, the DMA controller transfers data directly into a memory buffer.

When a block of data is transferred, the DMA controller can generate an interrupt to inform the CPU.

The CPU then processes the completed buffer while the DMA controller continues filling another buffer.

This can be implemented using double buffering.

Acquisition Device



DMA



Buffer A



CPU processes A

Meanwhile:

Acquisition Device



DMA



Buffer B

This allows continuous data acquisition.

DMA Transfer vs Interrupt-Driven I/O

Feature	DMA	Interrupt-Driven I/O
CPU involvement	Very low	High
Suitable for 500 MB/s	Yes	Generally unsuitable
Data transfer	Device ↔ Memory	Device → CPU → Memory
Interrupt frequency	Low, usually per block	Potentially very high
CPU overhead	Low	High
Continuous streaming	Excellent	Less efficient

Analytical Comparison

At 500 MB/s, suppose data is transferred in 4 KB blocks.

The approximate number of blocks per second is:

If an interrupt were generated for every 4 KB block, the CPU would need to handle approximately 122,000 interrupts per second.

This would create significant CPU overhead.

With DMA, larger blocks can be transferred and an interrupt can be generated only after a larger buffer is completed. Therefore, the interrupt rate is significantly reduced.

Memory-Mapped I/O

The acquisition device's control and status registers are mapped into the processor's memory address space.

For example:

Control Register → 0x9000

Status Register → 0x9004

Data Register → 0x9008

DMA Register → 0x9010

The CPU accesses these registers using normal memory instructions.

Justification

DMA is clearly preferable for a continuous 500 MB/s data stream because it removes the CPU from the bulk data path.

Interrupt-driven I/O would require frequent CPU intervention and could consume a large portion of CPU resources.

Memory-mapped I/O is used for efficient configuration and status monitoring, while DMA handles the actual high-speed data movement.

Constraint Satisfaction

Constraint	Proposed Solution
Continuous 500 MB/s stream	High-speed DMA transfer
Minimal CPU intervention	DMA handles bulk transfer
Compare DMA and interrupt-driven I/O	DMA has much lower CPU overhead
Device registers	Memory-mapped I/O
Continuous operation	Double-buffered memory
Transfer notification	DMA completion interrupt

Conclusion

For a continuous 500 MB/s data stream, DMA is the preferred transfer mechanism. It

minimizes CPU intervention and provides efficient continuous data movement. Memory-mapped I/O provides efficient access to device registers, while completion interrupts notify the CPU only when processing is required.

Q5. Interrupt Handling Mechanism

Problem Understanding

The objective is to construct an interrupt handling mechanism that supports nested interrupts and guarantees a maximum interrupt latency of less than 10 μ s.

High-priority devices must use vectored interrupts so that their interrupt service routines can be accessed quickly. Software interrupts should be used for background tasks that do not require immediate hardware response.

The design must therefore provide interrupt prioritization, nesting, fast vector dispatch and efficient background task handling.

Constraint Analysis

The most important requirement is:

Interrupt latency is the time between an interrupt being generated and the beginning of its corresponding interrupt service routine.

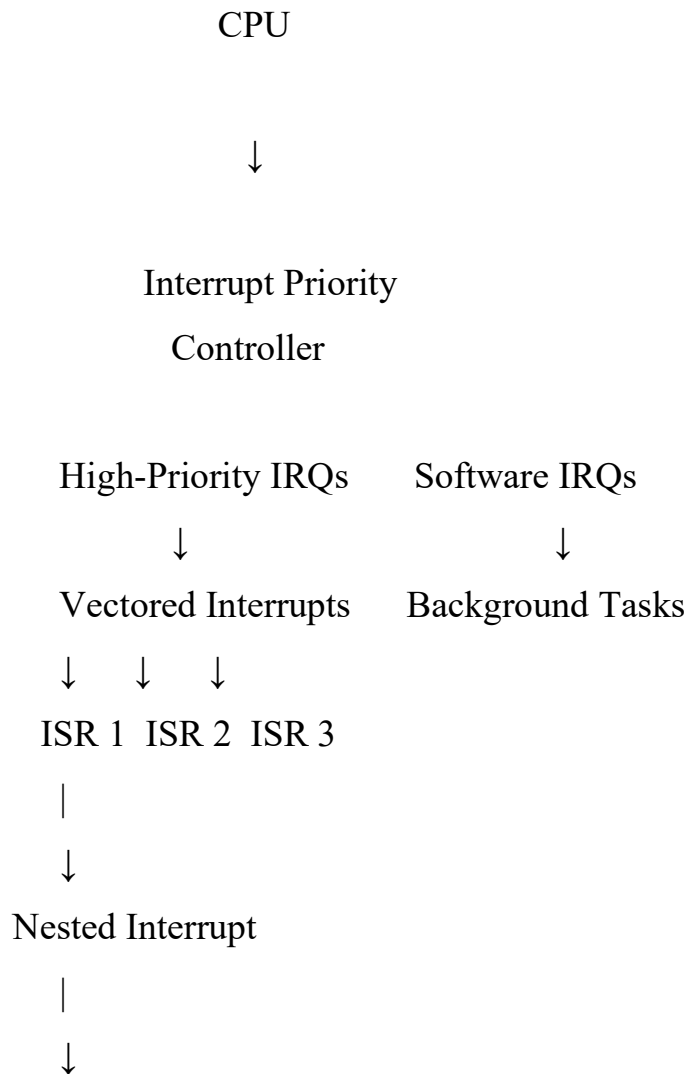
High-priority interrupts must therefore be serviced quickly.

Nested interrupts are required so that a high-priority interrupt can interrupt a lower-priority ISR.

Vectored interrupts should be assigned to critical devices so that the CPU can directly access the appropriate ISR.

Software interrupts are suitable for background tasks because they do not require the same immediate response as critical hardware events.

Proposed Interrupt Architecture



Higher Priority ISR

Working of the Proposed System

When an interrupt occurs, the interrupt controller determines its priority.

If it is a high-priority interrupt, the CPU immediately saves the current execution context and jumps to the corresponding vectored ISR.

If another higher-priority interrupt occurs while a lower-priority ISR is executing, the current ISR can be temporarily suspended.

The CPU services the higher-priority ISR first.

After completion, the CPU restores the previous context and continues the interrupted ISR.

Low-Priority ISR starts



High-Priority interrupt occurs



Save current context



Execute high-priority ISR



Return from high-priority ISR



Restore previous context



Continue low-priority ISR

Nested Interrupt Handling

Nested interrupts allow a high-priority interrupt to interrupt a lower-priority ISR.

For example:

Priority 1 → Critical device

Priority 2 → Normal device

Priority 3 → Background task

If Priority 3 is executing and Priority 1 occurs:

Background ISR



Priority 1 interrupt



Priority 1 ISR



Return



Background ISR continues

This ensures that critical events are not delayed by less important tasks.

Interrupt Latency Analysis

The required maximum interrupt latency is:

The system should minimize:

- Interrupt detection time
- Context-saving time
- Vector lookup time
- ISR entry time

A high-priority interrupt should therefore have a dedicated vector and a short ISR.

The design target can be represented as:

The total must satisfy:

The system should be tested under worst-case interrupt load to verify that this requirement is satisfied.

Vectored Interrupts for High-Priority Devices

High-priority devices receive dedicated interrupt vectors.

Critical Device 1 → Vector 1 → ISR 1

Critical Device 2 → Vector 2 → ISR 2

Critical Device 3 → Vector 3 → ISR 3

This avoids unnecessary polling and reduces the time required to identify the interrupt source.

Software Interrupts for Background Tasks

Software interrupts are used for tasks that do not require immediate hardware response.

Examples include:

- Deferred processing
- Operating system services
- Background communication
- Data processing
- Driver-level software requests

This prevents background operations from interfering with high-priority real-time interrupts.

Interrupt Priority Structure

A priority-based structure can be used:

Highest Priority



Critical Hardware Interrupts



Normal Hardware Interrupts



Software Interrupts



Background Tasks

Lowest Priority

This ensures that critical events receive CPU attention before background tasks.

Justification

Nested interrupts allow urgent events to interrupt less important operations. Vectored interrupts reduce the time needed to identify high-priority interrupt sources. Short ISRs

reduce the amount of time the CPU remains inside interrupt handling.

Software interrupts are placed at lower priority because background tasks do not require the same strict latency as critical hardware events.

Together, these techniques allow the system to target the required maximum interrupt latency of less than 10 μ s.

Constraint Satisfaction

Constraint	Proposed Solution
Support nested interrupts	Higher-priority ISR can interrupt lower-priority ISR
Maximum latency < 10 μ s	Fast vector dispatch + short high-priority ISR
Vectored interrupts	Dedicated vectors for critical devices
Software interrupts	Used for background/deferred tasks
Priority handling	Interrupt priority controller
Fast response	Context saving + direct ISR execution

Conclusion

The proposed interrupt mechanism combines nested interrupts, priority-based interrupt handling, vectored interrupts and software interrupts. High-priority devices receive immediate service through dedicated vectors, while background tasks are handled using software interrupts. By minimizing interrupt entry overhead and keeping critical ISRs short, the system can be designed and verified to meet the required less-than-10- μ s interrupt latency.